AI ENGINEERING INTERNSHIP

Daily Task Report

Days 1 · 2 · 3 — Technical Documentation

01	02	03
Data Engineering & Ingestion Pipeline	Rule-Based NLP & Intent Classification	Text Processing & Search Engine

Technologies Used: Python · REST APIs · Regex · NLP · JSON · dotenv

Confidential — Internal Internship Documentation

Table of Contents

Day 1 — Data Engineering & Live Weather Pipeline Page 3

1.1 Objective | 1.2 Architecture Overview | 1.3 Technical Deep Dive | 1.4 Output Schema | 1.5 Full Code

Day 2 — Rule-Based NLP & News Classification Engine Page 7

2.1 Objective | 2.2 System Architecture | 2.3 NewsClassifier Class | 2.4 Entity Extraction | 2.5 Full Code

Day 3 — Keyword Search & Jaccard Ranking Engine Page 12

3.1 Objective | 3.2 NLP Pipeline | 3.3 Jaccard Similarity | 3.4 Synonym Expansion | 3.5 Full Code

DAY 1 · TASK 01

Data Engineering & Live Weather Ingestion Pipeline

1.1 Objective

The goal of Day 1 was to build a **resilient, production-style data pipeline** that interfaces with the **OpenWeatherMap REST API**, fetches live meteorological data for 10 Indian cities, and transforms the raw nested JSON response into a flat, analysis-ready schema.

This task covers three core data engineering concepts: **API integration**, **data transformation**, and **structured error handling** — skills that form the backbone of any real-world ETL (Extract, Transform, Load) workflow.

1.2 Architecture Overview

```
[ .env File ]
     │  API Key (secure, not hardcoded)
     ▼
[ load_dotenv() ] ──► os.getenv("OPENWEATHER_API_KEY")
     │
     ▼
[ CITIES list ] ──► 10 Indian cities as input
     │
     ▼ (loop)
[ process_city(city) ]
     │
     ├──► Build URL ──► requests.get()
     │
     ├──► Status Check: 404 → City Not Found
     │                  401 → Invalid API Key
     │                  other → raise_for_status()
     │
     ├──► Parse JSON ──► Flatten schema
     │         temp: K → °C  |  dt: Unix → ISO datetime
```

```
      │
      └──► Return city_data dict  OR  None on failure
      │
      ▼
[ results list ] ──► json.dump() ──► weather_data.json
```

1.3 Technical Deep Dive

API Key Security — Environment Variables

Hardcoding an API key directly in source code is a serious security vulnerability. If the file is ever pushed to GitHub, the key is exposed publicly.

The correct approach used here is storing the key in a `.env` file and reading it at runtime:

```
from dotenv import load_dotenv
load_dotenv()
API_KEY = os.getenv("OPENWEATHER_API_KEY")
```

The `.env` file is kept **out of version control** (added to `.gitignore`). The `python-dotenv` library loads it into environment variables on startup. This is standard practice in all professional Python projects.

Error Handling — Defensive Programming

The `process_city()` function uses a layered error handling strategy:

```
if resp.status_code == 404:
    print(f"City not found: {city}")
    return None

if resp.status_code == 401:
    print("Invalid API Key. Please check your .env file.")
    return None

resp.raise_for_status()
```

Why this matters: The loop never crashes. If one city name is misspelled or the API key expires mid-run, that single call returns `None` and the program continues to the next city. The `raise_for_status()` at the end acts as a catch-all for any other HTTP error codes (500, 503, etc.).

Unit Transformation — Kelvin to Celsius

The OpenWeatherMap API returns temperatures in **Kelvin** by default. Kelvin is an absolute temperature scale (0 K = absolute zero, −273.15°C). The conversion formula is:

$$°C = K - 273.15$$

In the code:

```
"temp_c": round(data["main"]["temp"] - 273.15, 2)
```

`round(..., 2)` keeps the output clean — two decimal places is sufficient precision for weather reporting.

Timestamp Conversion — Unix Epoch to ISO Format

APIs commonly return timestamps as **Unix Epoch time** — the number of seconds elapsed since January 1, 1970, 00:00:00 UTC. This is compact and timezone-neutral, but not human-readable.

```
"last_updated": datetime.fromtimestamp(data["dt"]).strftime('%Y-%m-%d %H:%M:%S')
```

`datetime.fromtimestamp()` converts the integer to a `datetime` object, and `.strftime()` formats it into the **ISO 8601** standard: `YYYY-MM-DD HH:MM:SS`.

Schema Flattening

The raw API response is deeply nested:

```
{
  "main": { "temp": 305.15, "humidity": 72 },
  "weather": [{ "main": "Haze", "description": "haze" }],
  "name": "Mumbai",
  "dt": 1719830400
}
```

Nested data is inconvenient to query or load into a database. The pipeline flattens it:

```
city_data = {
    "city":         data["name"],
    "temp_c":       round(data["main"]["temp"] - 273.15, 2),
    "humidity":     data["main"]["humidity"],
    "weather":      data["weather"][0]["main"],
    "last_updated": datetime.fromtimestamp(data["dt"]).strftime('%Y-%m-%d %H:%M:%S')
}
```

Every value is now at the **top level** — flat, consistent, and ready for direct analysis or database insertion.

1.4 Output Schema

Field	Source Path	Transformation
`city`	`data["name"]`	None — raw string

`temp_c`	`data["main"]["temp"]`	`K - 273.15`, rounded to 2 decimal places
`humidity`	`data["main"]["humidity"]`	None — percentage integer
`weather`	`data["weather"][0]["main"]`	Extracted from list index 0
`last_updated`	`data["dt"]`	Unix epoch → `YYYY-MM-DD HH:MM:SS`

1.5 Full Source Code

```
import os
import requests
import json
from datetime import datetime
from dotenv import load_dotenv

load_dotenv()
API_KEY = os.getenv("OPENWEATHER_API_KEY")

CITIES = [
    "Mumbai", "Delhi", "Bengaluru", "Hyderabad", "Chennai",
    "Kolkata", "Ahmedabad", "Pune", "Jaipur", "Lucknow"
]

def process_city(city):
    try:
        url = f"https://api.openweathermap.org/data/2.5/weather?q={city}&appid={API_KEY}"
        resp = requests.get(url)

        if resp.status_code == 404:
            print(f"City not found: {city}")
            return None

        if resp.status_code == 401:
            print("Invalid API Key. Please check your .env file.")
            return None

        resp.raise_for_status()
        data = resp.json()

        city_data = {
            "city": data["name"],
            "temp_c": round(data["main"]["temp"] - 273.15, 2),
            "humidity": data["main"]["humidity"],
            "weather": data["weather"][0]["main"],
            "last_updated": datetime.fromtimestamp(data["dt"]).strftime('%Y-%m-%d %H:%M:%S')
        }

        print(f"Done: {city_data['city']} | {city_data['temp_c']}°C |
{city_data['weather']}")
```

```python
        return city_data

    except Exception as e:
        print(f"Error for {city}: {e}")
        return None


if __name__ == "__main__":
    if not API_KEY or API_KEY == "your_api_key_here":
        print("Please update your .env file with your API key.")
    else:
        results = []
        for city in CITIES:
            city_data = process_city(city)
            if city_data is not None:
                results.append(city_data)

        with open("weather_data.json", "w") as f:
            json.dump(results, f, indent=4)

        print(f"\nSaved weather data for {len(results)} cities to weather_data.json")
```

DAY 2 · TASK 02

Rule-Based NLP Engine & Live News Classification

2.1 Objective

Day 2 introduces **Natural Language Processing (NLP)** concepts applied to a live data stream. The task was to connect to the **NewsData.io API**, fetch up to 50 current news headlines, and build a deterministic **rule-based classification engine** that:

1. Categorises each headline into `FINANCE`, `TECH`, `POLITICS`, or `GENERAL`
2. Extracts structured entities (companies, currencies, percentages) using **Regular Expressions**
3. Uses a **modular JSON config** for keyword management — no hardcoded logic

The critical constraint: **Zero LLM usage**. Every classification decision must be explainable, deterministic, and cost-free. This simulates real-world scenarios where speed and zero inference cost are priorities.

2.2 System Architecture

```
[ .env File ]
     │  NEWSDATA_API_KEY
     ▼
[ fetch_news(keyword, count=50) ]
     │
     ├──► GET https://newsdata.io/api/1/latest?q=...
     │
     ├──► Status 429? ──► time.sleep(5) ──► Retry once
     │
     └──► Return articles list (up to 50)
                │
                ▼ (for each article)
[ NewsClassifier ] ◄── news_config.json (keyword map)
     │
     ├──► classify(full_text)
     │         lowercase → keyword match → score each
```

```
category
     │         → return category with highest score
     │
     └──► extract_entities(full_text)
              ├── Companies (known list match)
              ├── Currency symbols (Regex)
              └── Percentages (Regex)
                   │
                   ▼
          [ news_data_day2.json ]
```

2.3 The NewsClassifier Class — Deep Explanation

Constructor: Loading Config at Runtime

```python
def __init__(self, config_path):
    base_dir = os.path.dirname(os.path.abspath(__file__))
    full_config_path = os.path.join(base_dir, config_path)
    with open(full_config_path, 'r') as f:
        self.config = json.load(f)
    self.categories = self.config['categories']
```

`os.path.dirname(os.path.abspath(__file__))` gets the **absolute directory** of the script, regardless of where it is run from. This prevents path errors when running from a different working directory — a common issue in professional setups.

The config is loaded **once** in the constructor and stored as `self.categories`. This is efficient — no repeated file I/O during classification.

Classification: Keyword-Weighted Scoring

```python
def classify(self, text):
    text = text.lower()
    scores = {}
    for cat, keywords in self.categories.items():
        count = 0
        for word in keywords:
            if word in text:
                count += 1
        scores[cat] = count
    best_cat = max(scores, key=scores.get)
    if scores[best_cat] == 0:
```

```
        return "GENERAL"
    return best_cat
```

How it works: The text is lowercased to ensure case-insensitive matching. Each category is scored by counting how many of its keywords appear in the text. The category with the **highest score wins**. If no keywords match at all (score = 0), it returns `"GENERAL"`.

This is a **multi-class, additive scoring** system. A headline mentioning `"market"`, `"inflation"`, and `"fiscal"` scores 3 for FINANCE vs 0 for TECH — a clear winner.

The Config File — Separation of Concerns

```
{
    "categories": {
        "FINANCE": ["economy", "finance", "stock", "market", "bank",
                    "interest", "inflation", "revenue", "fiscal", "gdp", "trade"],
        "TECH":    ["technology", "software", "ai", "artificial intelligence",
                    "hardware", "internet", "crypto", "bitcoin", "startup", "innovation"],
        "POLITICS":["government", "election", "policy", "president", "senate",
                    "law", "vote", "summit", "minister", "parliament", "treaty"]
    }
}
```

Keeping keywords in a separate JSON file is a **software engineering best practice** called **Separation of Concerns**. The classification logic (Python) is completely decoupled from the configuration data (JSON). To add a new category or keyword, you edit the JSON file — the Python code never needs to change. This makes the system maintainable and scalable.

2.4 Entity Extraction — Regular Expressions Explained

Company Detection — Known Entity List

```
known_companies = [
    "Apple", "Google", "Microsoft", "Tesla", "Amazon", "Meta",
    "Netflix", "Twitter", "Samsung", "Intel", "Nvidia", "IBM",
    "Goldman Sachs", "JPMorgan", "BlackRock", "Alibaba", "Tata",
    "Reliance", "Infosys", "Wipro", "HDFC", "Adani"
]
for company in known_companies:
    if company.lower() in text.lower():
        found_companies.append(company)
```

A simple regex like `[A-Z][a-z]+` for company detection would match every capitalised word — every sentence start, every month name, every country. That produces noise, not signal. Using a **curated known-entities list** is more precise and reflects how real NER (Named Entity Recognition) systems build domain-specific entity lists.

Currency Symbol Extraction — Regex

```
"Currency": list(set(re.findall(r'[\$\£\€\₹¥]', text)))
```

The pattern `[\$\£\€\₹¥]` is a **character class** — it matches any single character that is one of those currency symbols. `re.findall()` returns all matches as a list. Wrapping with `set()` removes duplicates (a text mentioning `$` five times returns `['$']` once).

Percentage Extraction — Regex

```
"Percentages": list(set(re.findall(r'\d+(?:\.\d+)?%', text)))
```

Breaking this pattern down:

Part	Meaning
`\d+`	One or more digits (e.g., `5`, `12`, `100`)
`(?:\.\d+)?`	Optionally followed by a decimal point and more digits (e.g., `.5`, `.75`)
`%`	Literal percent sign

This matches `5%`, `12.5%`, `100%`, `3.14%` — all valid percentage formats in news text.

2.5 Rate Limiting — Exponential Backoff Concept

```
if response.status_code == 429:
    print("Rate limit hit. Waiting 5 seconds before retrying...")
    time.sleep(5)
    response = requests.get(url)
```

HTTP **429 Too Many Requests** means the API server is throttling the client. The solution implemented here is a **simple retry with delay**. In production systems, this is extended to **exponential backoff** — doubling the wait time on each failure (5s, 10s, 20s...) to avoid overwhelming the server on recovery.

2.6 Full Source Code

```
import os, requests, json, re, time
from dotenv import load_dotenv

load_dotenv()
API_KEY = os.getenv("NEWSDATA_API_KEY")

class NewsClassifier:
    def __init__(self, config_path):
        base_dir = os.path.dirname(os.path.abspath(__file__))
        full_config_path = os.path.join(base_dir, config_path)
        with open(full_config_path, 'r') as f:
            self.config = json.load(f)
```

```python
        self.categories = self.config['categories']

    def classify(self, text):
        text = text.lower()
        scores = {}
        for cat, keywords in self.categories.items():
            count = 0
            for word in keywords:
                if word in text:
                    count += 1
            scores[cat] = count
        best_cat = max(scores, key=scores.get)
        if scores[best_cat] == 0:
            return "GENERAL"
        return best_cat

    def extract_entities(self, text):
        known_companies = [
            "Apple","Google","Microsoft","Tesla","Amazon","Meta",
            "Netflix","Twitter","Samsung","Intel","Nvidia","IBM",
            "Goldman Sachs","JPMorgan","BlackRock","Alibaba","Tata",
            "Reliance","Infosys","Wipro","HDFC","Adani"
        ]
        found_companies = [c for c in known_companies if c.lower() in text.lower()]
        return {
            "Companies":   found_companies,
            "Currency":    list(set(re.findall(r'[\$\£\€\₹¥]', text))),
            "Percentages": list(set(re.findall(r'\d+(?:\.\d+)?%', text)))
        }

def fetch_news(keyword, count=50):
    url = f"https://newsdata.io/api/1/latest?apikey={API_KEY}&q={keyword}"
    try:
        response = requests.get(url)
        if response.status_code == 429:
            print("Rate limit hit. Waiting 5 seconds...")
            time.sleep(5)
            response = requests.get(url)
        response.raise_for_status()
        return response.json().get("results", [])[:count]
    except Exception as e:
        print(f"Error: {e}")
        return []

if __name__ == "__main__":
    classifier = NewsClassifier("news_config.json")
    articles = fetch_news("Global Economy", count=50)
    processed_news = []
    for art in articles:
        title = art.get("title", "")
        desc  = art.get("description", "") or ""
        full_text = f"{title} {desc}"
```

```
    processed_news.append({
        "title":    title,
        "category": classifier.classify(full_text),
        "entities": classifier.extract_entities(full_text),
        "link":     art.get("link")
    })
    print(f"[{processed_news[-1]['category']}] {title[:60]}...")
base_dir = os.path.dirname(os.path.abspath(__file__))
with open(os.path.join(base_dir, "news_data_day2.json"), "w") as f:
    json.dump(processed_news, f, indent=4)
print(f"\nDone. {len(processed_news)} articles saved.")
```

DAY 3 · TASK 03

Text Processing & Keyword-Based Search Ranking Engine

3.1 Objective

Day 3 moves deeper into **NLP text processing**. The task was to build a **Mini Search Engine** from scratch — without any external NLP libraries (no NLTK, no spaCy, no transformers). The engine must:

1. **Preprocess** raw text (lowercase, remove punctuation, filter stop words)
2. **Tokenise** sentences into meaningful word sets
3. **Rank** each sentence in a 50-sentence corpus against a user query using **Jaccard Similarity**
4. **Expand queries** with synonyms to improve recall *(Stretch Task)*

The corpus contains a deliberate mix of **signal** (AI/tech sentences) and **noise** (unrelated topics like gardening, desserts, Roman history), testing whether the ranking algorithm can correctly surface relevant results.

3.2 The NLP Preprocessing Pipeline

Text preprocessing is the most important step in any NLP system. Raw text contains noise that confuses similarity algorithms — uppercase letters, punctuation, common words like "the" and "is" that carry no semantic meaning.

Step 1 — Lowercasing

```
text = text.lower()
```

Converts `"LLM Training"` → `"llm training"`. Without this, `"LLM"` and `"llm"` would be treated as completely different tokens. Case normalisation is always the first step in text preprocessing.

Step 2 — Punctuation Removal

```
text = text.translate(str.maketrans("", "", string.punctuation))
```

`string.punctuation` is a Python constant containing all punctuation characters: `!"#$%&'()*+,-./:;<=>?@[\]^_{|}~`. `str.maketrans("", "", ...)` creates a **translation table** that maps each punctuation character to `None` (delete). `.translate()` applies it. This is faster than using `re.sub()` for this specific case.

Step 3 — Stop Word Filtering

```
STOP_WORDS = { "the", "is", "at", "a", "an", "and", "or", ... }
tokens = [word for word in tokens if word not in STOP_WORDS]
```

Stop words are high-frequency words that appear in almost every sentence but carry no semantic meaning. Words like "the", "is", "and", "of" would inflate the overlap between any two sentences. By removing them, only **meaningful content words** remain for comparison.

Using a Python `set` for `STOP_WORDS` is a performance decision — set lookup is **O(1)** (constant time) vs list lookup at **O(n)**. For large corpora, this makes a significant difference.

3.3 Jaccard Similarity — The Ranking Algorithm

Jaccard Similarity is a statistical measure of **set overlap**. It answers: "How similar are two sets?"

$$\text{Jaccard}(A, B) = |A \cap B| / |A \cup B|$$

Intersection size divided by Union size · Range: 0.0 (no overlap) to 1.0 (identical)

```
def jaccard_similarity(set_a, set_b):
    if not set_a or not set_b:
        return 0.0
    intersection = len(set_a & set_b)
    union = len(set_a | set_b)
    return intersection / union
```

Worked Example

Query tokens	`{"llm", "training", "deep", "learning"}`
Sentence tokens	`{"nvidia", "new", "gpu", "architecture", "llm", "training"}`
Intersection	`{"llm", "training"}` → size = **2**
Union	`{"nvidia", "new", "gpu", "architecture", "llm", "training", "deep", "learning"}` → size = **8**
Score	2 / 8 = **0.25**

A score of `0.25` would rank this sentence near the top among the 50-sentence corpus, since most unrelated sentences would score `0.0000`.

3.4 Synonym Expansion — Stretch Task

Standard keyword search is **exact-match only**. If a user searches `"happy"` but the corpus contains `"joyful"`, a basic engine misses it. Query expansion solves this.

```
SYNONYMS = {
    "happy": ["joyful", "glad", "pleased", "cheerful"],
```

```
    "ai":    ["artificial intelligence", "machine learning", "deep learning"],
    "llm":   ["large language model", "language model", "gpt", "bert"],
    ...
}

def expand_query(tokens):
    expanded = set(tokens)
    for token in tokens:
        if token in SYNONYMS:
            for synonym in SYNONYMS[token]:
                for word in synonym.split():
                    expanded.add(word)
    return expanded
```

Multi-word synonyms like `"artificial intelligence"` are split into individual tokens (`"artificial"` , `"intelligence"`) so they can match against individual words in sentence token sets. The result is a **significantly larger query set** that improves recall without changing the core Jaccard formula.

Example expansion:

```
Input query:     "LLM training"
After tokens:    ["llm", "training"]
After expand:    {"llm", "training", "large", "language", "model",
                  "language", "gpt", "bert", "train", "fine-tune", "study"}
```

3.5 Signal vs. Noise — What the Engine Demonstrates

The corpus was deliberately designed with 50 sentences, most being **noise** (gardening, hiking, Roman history, baking bread). Running the query `"LLM training deep learning"` should return AI-related sentences at the top and push noise sentences to score `0.0000` .

This tests whether the preprocessing + Jaccard pipeline works correctly — the key metric being: **can signal reliably beat noise in the rankings?**

3.6 Full Source Code

```
import os, string

STOP_WORDS = {
    "the","is","at","a","an","and","or","but","in","on","of","to","for",
    "with","by","from","it","its","this","that","are","was","were","be",
    "been","have","has","had","do","does","did","not","as","their","they",
    "we","he","she","i","you","my","our","can","will","would","could",
    "should","one","also","into","more","most","than","so","if","about",
    "which","when","who","how","what"
}

SYNONYMS = {
    "happy":  ["joyful","glad","pleased","cheerful"],
    "sad":    ["unhappy","sorrowful","depressed","upset"],
```

```
    "fast":   ["quick","rapid","swift","speedy"],
    "big":    ["large","huge","massive","enormous"],
    "smart":  ["intelligent","clever","bright"],
    "ai":     ["artificial intelligence","machine learning","deep learning"],
    "llm":    ["large language model","language model","gpt","bert"],
    "learn":  ["train","fine-tune","study"],
    "data":   ["dataset","corpus","information"],
    "code":   ["programming","script","software","development"],
}

def preprocess_text(text):
    text = text.lower()
    text = text.translate(str.maketrans("","",string.punctuation))
    tokens = text.split()
    return [w for w in tokens if w not in STOP_WORDS]

def expand_query(tokens):
    expanded = set(tokens)
    for token in tokens:
        if token in SYNONYMS:
            for synonym in SYNONYMS[token]:
                for word in synonym.split():
                    expanded.add(word)
    return expanded

def jaccard_similarity(set_a, set_b):
    if not set_a or not set_b:
        return 0.0
    return len(set_a & set_b) / len(set_a | set_b)

def load_corpus(filepath):
    with open(filepath, "r", encoding="utf-8") as f:
        return [line.strip() for line in f if line.strip()]

def search(query, corpus_path="corpus.txt", top_n=5):
    sentences = load_corpus(corpus_path)
    query_tokens  = preprocess_text(query)
    query_expanded = expand_query(query_tokens)
    print(f"\nQuery: {query}")
    print(f"Expanded tokens: {sorted(query_expanded)}")
    print("-" * 65)
    results = []
    for sentence in sentences:
        score = jaccard_similarity(query_expanded, set(preprocess_text(sentence)))
        results.append((score, sentence))
    results.sort(key=lambda x: x[0], reverse=True)
    for rank, (score, sentence) in enumerate(results[:top_n], 1):
        print(f"  {rank}. [Score: {score:.4f}] {sentence}")

if __name__ == "__main__":
    base_dir = os.path.dirname(os.path.abspath(__file__))
    corpus_file = os.path.join(base_dir, "corpus.txt")
```

```python
test_queries = [
    "LLM training deep learning",
    "Happy joyful emotion",
    "API security GitHub",
    "data cleaning preprocessing",
]
for query in test_queries:
    search(query, corpus_path=corpus_file, top_n=3)
    print("=" * 65)
```

Summary — Skills Demonstrated Across 3 Days

Day 1 — Data Engineering

REST API integration · HTTP error handling · Unit conversion (Kelvin→Celsius) · Unix epoch formatting · JSON schema flattening · Secure credential management (.env)

Day 2 — NLP & Classification

OOP class design · Config-driven architecture · Keyword-weighted scoring · Regex entity extraction · Rate limit handling · Separation of concerns

Day 3 — Search & Ranking

Text preprocessing pipeline · Stop word filtering · Tokenisation · Jaccard Similarity algorithm · Set theory operations · Query synonym expansion

Cross-Cutting Practices

Zero external NLP dependencies · Modular function design · Defensive error handling · No hardcoded secrets · Clean, readable code structure